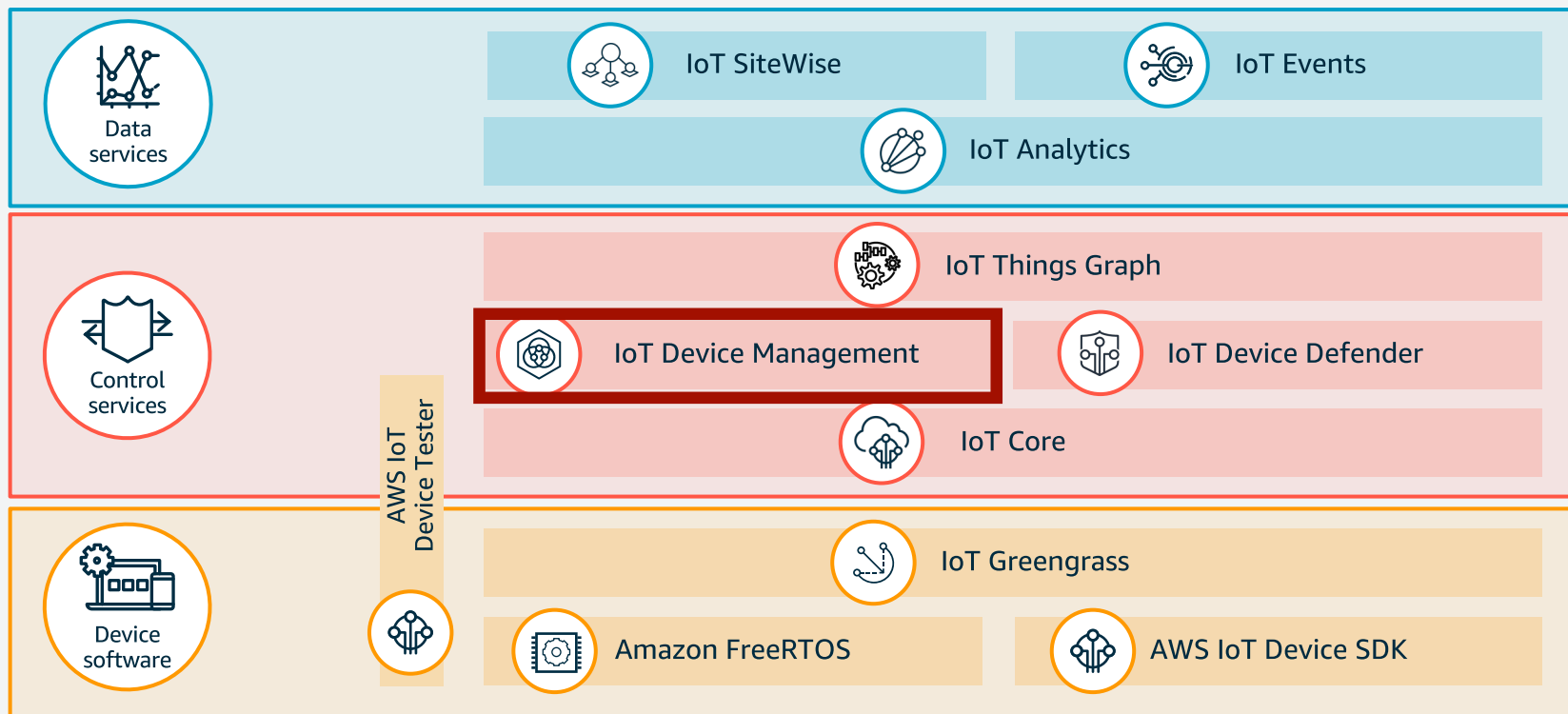




AWS IoT

Device Management Workshop

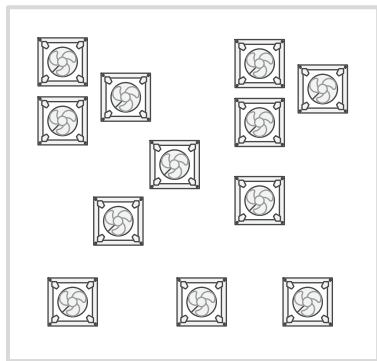
AWS IoT architecture



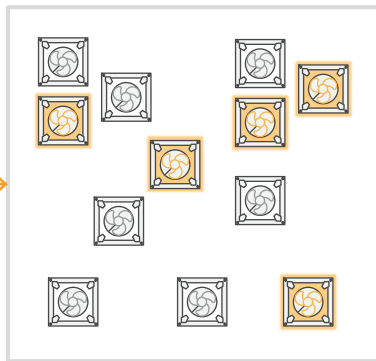


AWS IoT Device Management

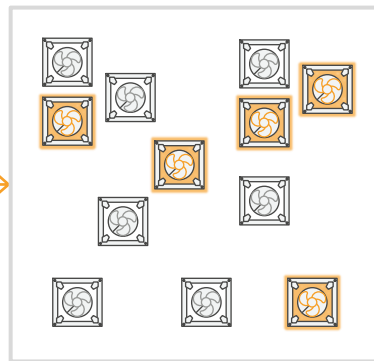
AWS IoT Device Management helps you register, organize, monitor, and remotely manage your growing fleet of connected devices.



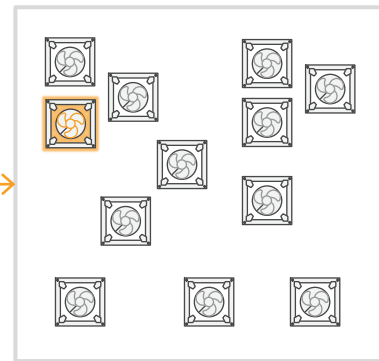
Fast device
registration at scale



Real-time fleet
indexing and search



Monitoring and
updating devices



Access individual device
securely



Connectivity
& Control
Services

Onboard

AWS IoT – starting to explore...

AWS IoT



Monitor

Activity

▼ Onboard

Get started

Fleet provisioning templates

► Manage

► Greengrass

► Secure

► Defend

► Act

Test

Software

Settings

Learn

Documentation

AWS IoT > Connect to AWS IoT

Connect to AWS IoT



Onboard a device

Connect a device or your computer to AWS IoT using the connection wizard for AWS IoT Device SDKs.

[Get started](#)



Onboard many devices

Start by creating a template that defines security policies and registry settings for your devices.

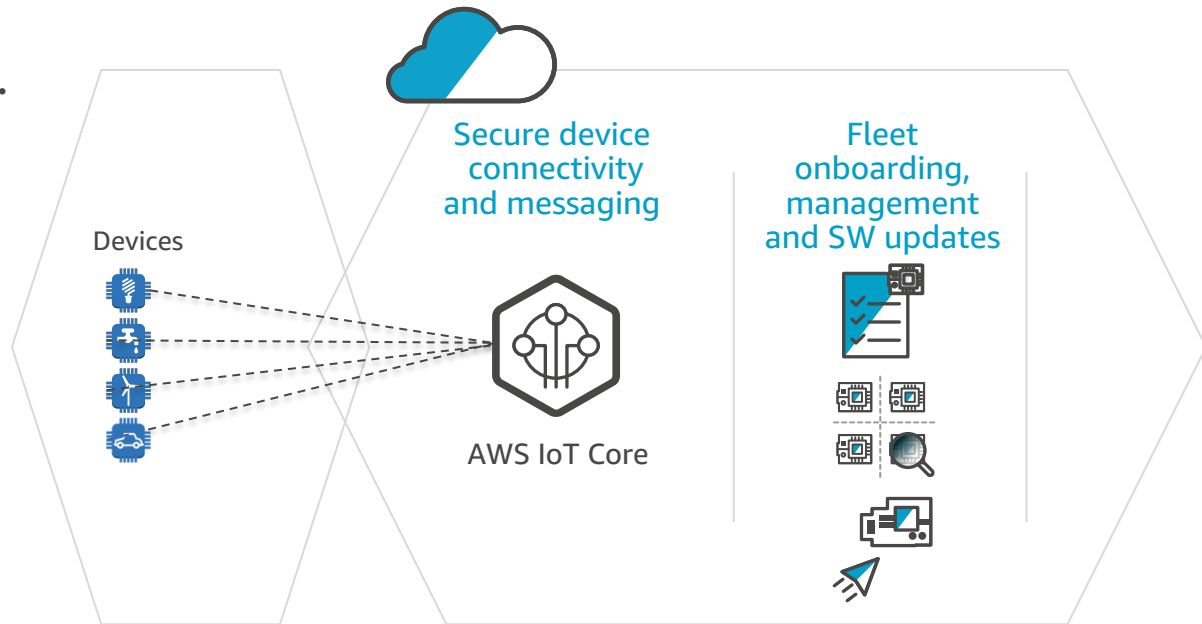
[Create template](#)

Did you know you can use the AWS SDK to work with data coming from your devices? [View the AWS SDKs](#)

At xcale - Howto provision devices?

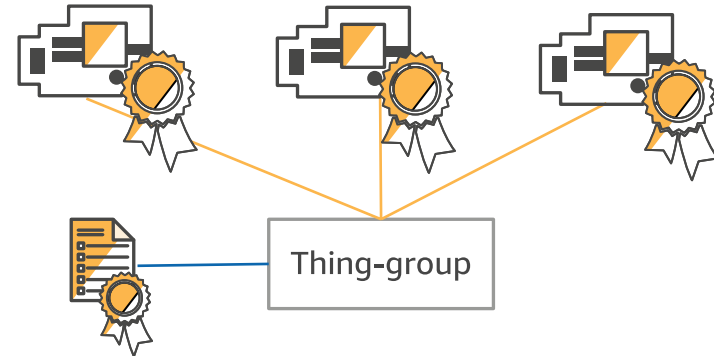
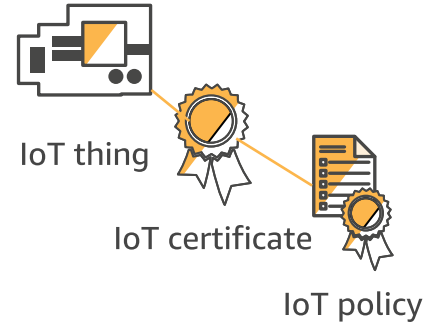
Architecture is developed...

How Do I onboard my devices???



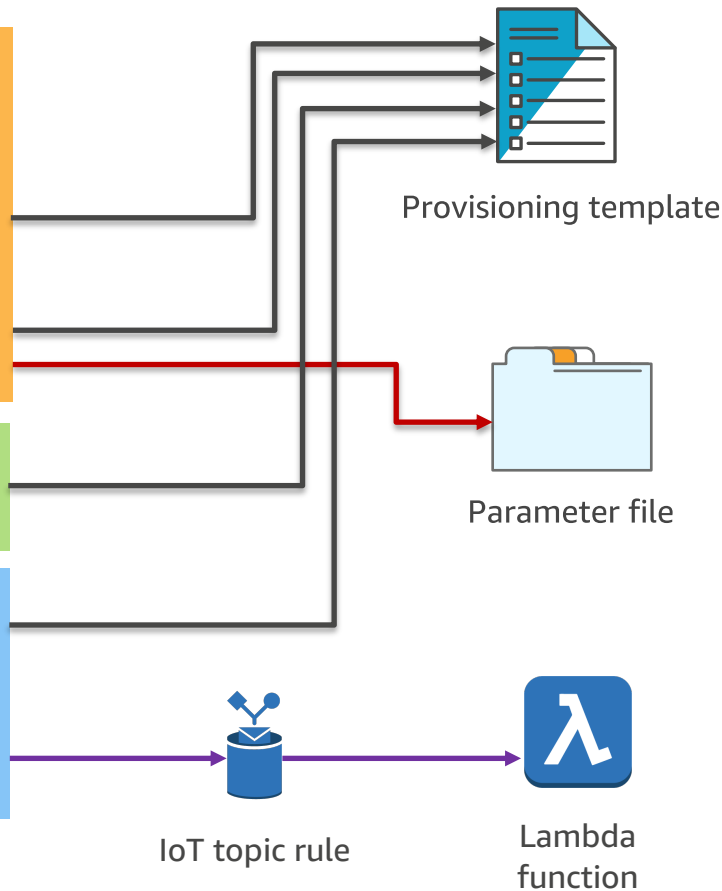
When a device is provisioned

- (Created in the device registry)
- Device certificate registered with AWS IoT Core
- (Certificate attached to the device)
- IoT Policy attached to the device through:
 - Certificate
 - Thing group



AWS IoT provisioning options

- API Calls
- Single Device Provisioning
- Bulk Device Provisioning
- Fleet Provisioning
- Just-in-Time Provisioning
- Just-in-Time Registration



Provisioning template

```

"Parameters": {
  "ThingName": {
    "Type": "AWS::IoT::Thing",
    "Properties": {
      "ThingName": {"Ref": "ThingName"},
      "AttributePayload": {
        "version": "v1",
        "serialNumber": {"Ref": "SerialNumber"}
      },
      "ThingTypeName": "lightBulb-versionA",
      "ThingGroups": ["v1-lightbulbs", {"Ref": "Location"}]
    }
  },
  "certificate": {
    "Type": "AWS::IoT::Certificate",
    "Properties": {
      "CertificateSigningRequest": {"Ref": "CSR"},
      "Status": "ACTIVE"
    }
  }
}

```

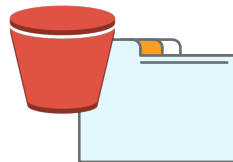
Single/Bulk device provisioning

```
{"ThingName": "foo", "SerialNumber": "123", "CSR": "csr1"} {"ThingName":  
"bar", "SerialNumber": "456", "CSR": "csr2"}
```

Parameters



Provisioning template



Parameter file
on S3 Bucket (bulk)

- Parameters with device information are used in the provisioning template
- Single: on "line" as parameter to register a thing
- Bulk: multiple parameter lines in an S3 bucket

Fleet provisioning: How it works

Use case 1

"Trusted bootstrap identity"

Device initial state



Device final state



Unique digital identity



IoT policy

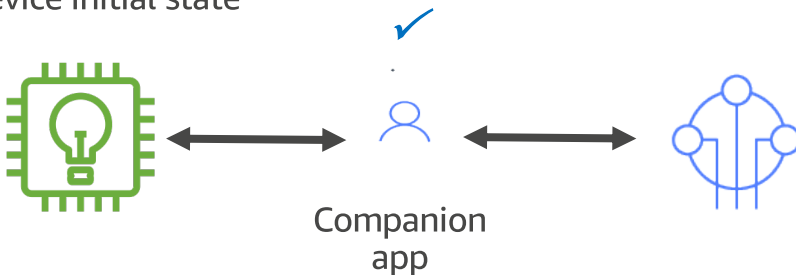


Thing registry (optional)

Use case 2

"Trusted user"

Device initial state



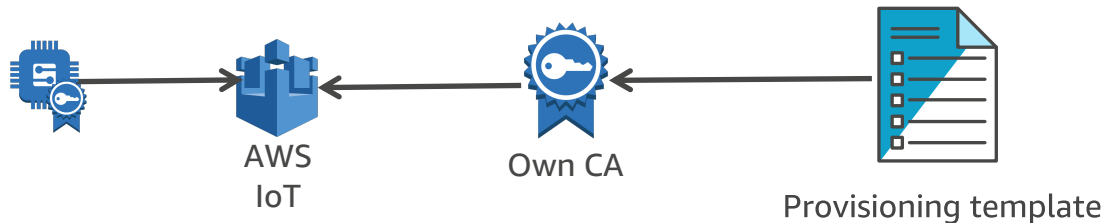
Fleet provisioning



Provisioning template

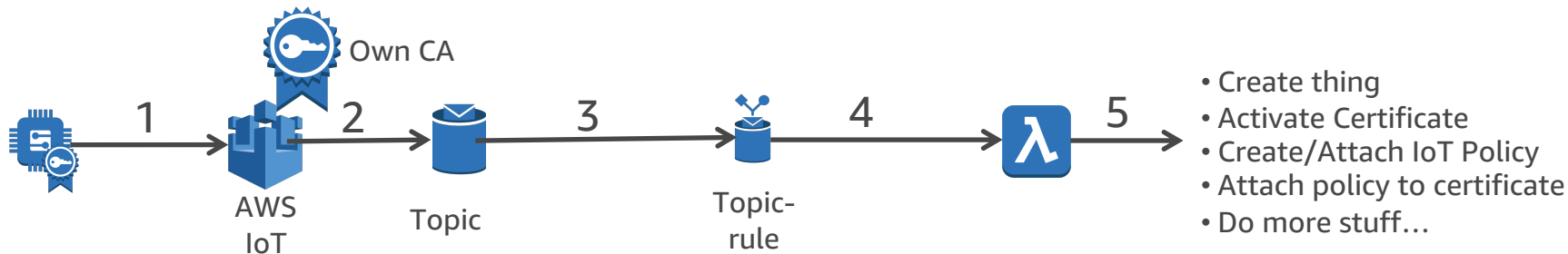
- Create a provisioning template
 - Optional Lambda-based pre-provisioning hook
- Create provisioning claim key/certificate
- Attach restricted policy to the claim certificate
- Device connects for the first time with claim key/certificate
- Uses the provisioning MQTT API to obtain final certificate and being provisioned in AWS IoT
 - `$aws/provisioning-templates/templateName/provision/payload-format`
 - `$aws/provisioning-templates/templateName/provision/payload-format/#`
 - `$aws/certificates/create-from-csr/payload-format/#`
 - `$aws/certificates/create/payload-format/#`

Device Onboarding – JITP



- Own CA required
 - Provisioning Template attached to own CA
1. Device connects to AWS IoT, device certificate gets registered
 2. JITP provisions device according to the provisioning template

Device Onboarding – JITR



1. Device connects to AWS IoT, device certificate gets registered

2. AWS IoT publishes message to
`$aws/events/certificates/registered/<caCertificateID>`

3. Topic Rule is invoked

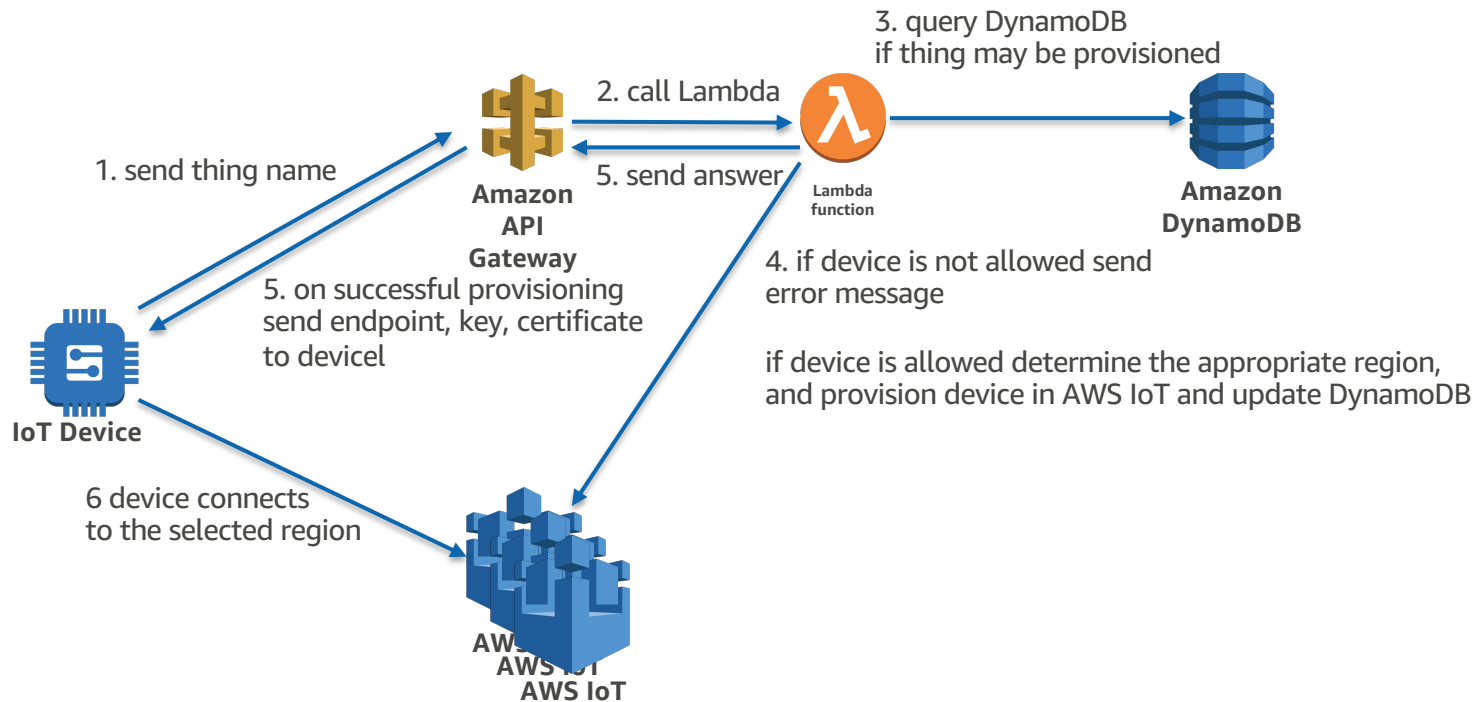
4. Topic Rule calls Lambda Function as action

5. Lambda provisions device

JITR vs. JITP

JITR	JITP
Topic rule and Lambda function. Code must be written and maintained	No code, only body template attached to CA
Provisioning more complex: Device connects, certificate registers with status PENDING_ACTIVATION, service sends MQTT message, rule triggers Lambda, Lambda does provisioning and optionally more stuff	Easy provisioning: Device connects, provisioning workflow run automatically
Flexible, different policies for different devices can be created/attached. Information from/to the provisioning process can be put/read from other systems, etc.	Static, same provisioning process for every device

Global Device Provisioning



Optional Exercise:

<https://aws.amazon.com/blogs/iot/provision-devices-globally-with-aws-iot/>

Organize



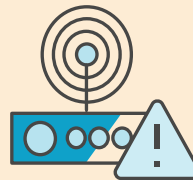
Grouping and Searching for Devices



Organize
into logical
Hierarchies



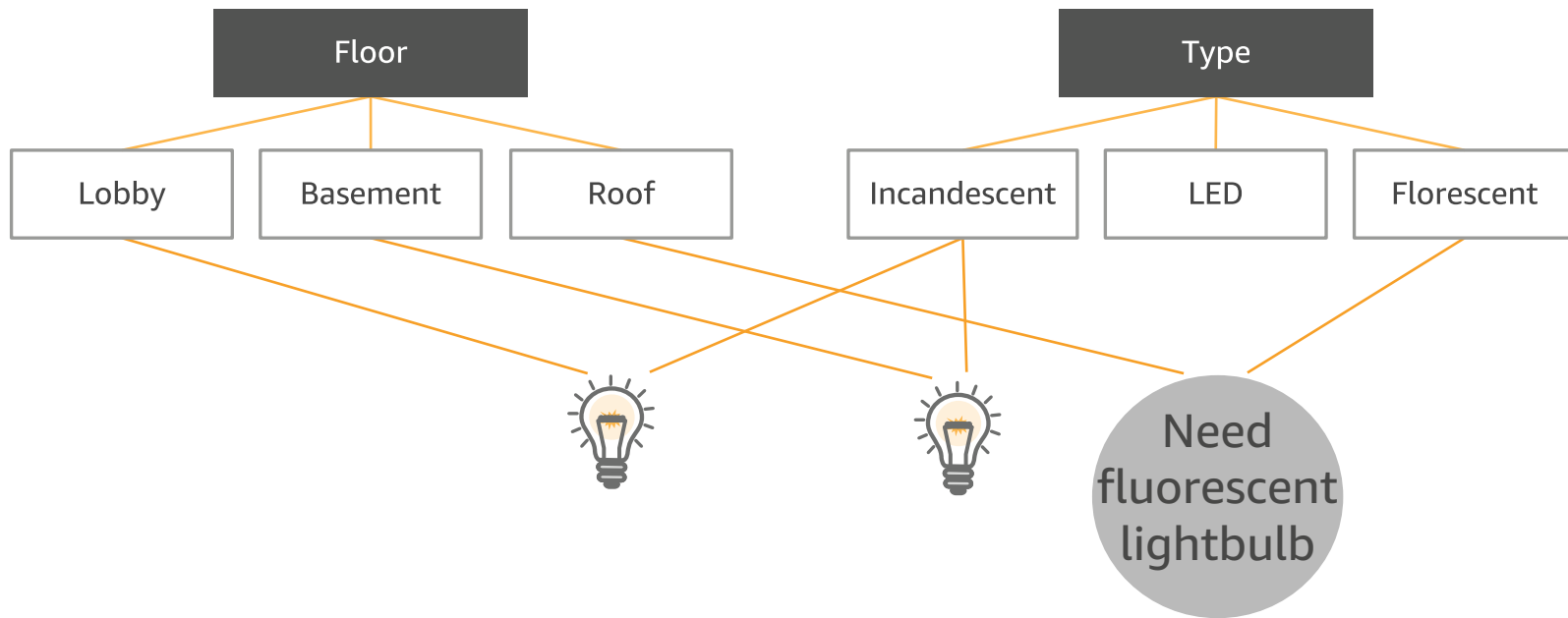
Search Both the
Registry and
Device Shadow



Notification of
Device Changes

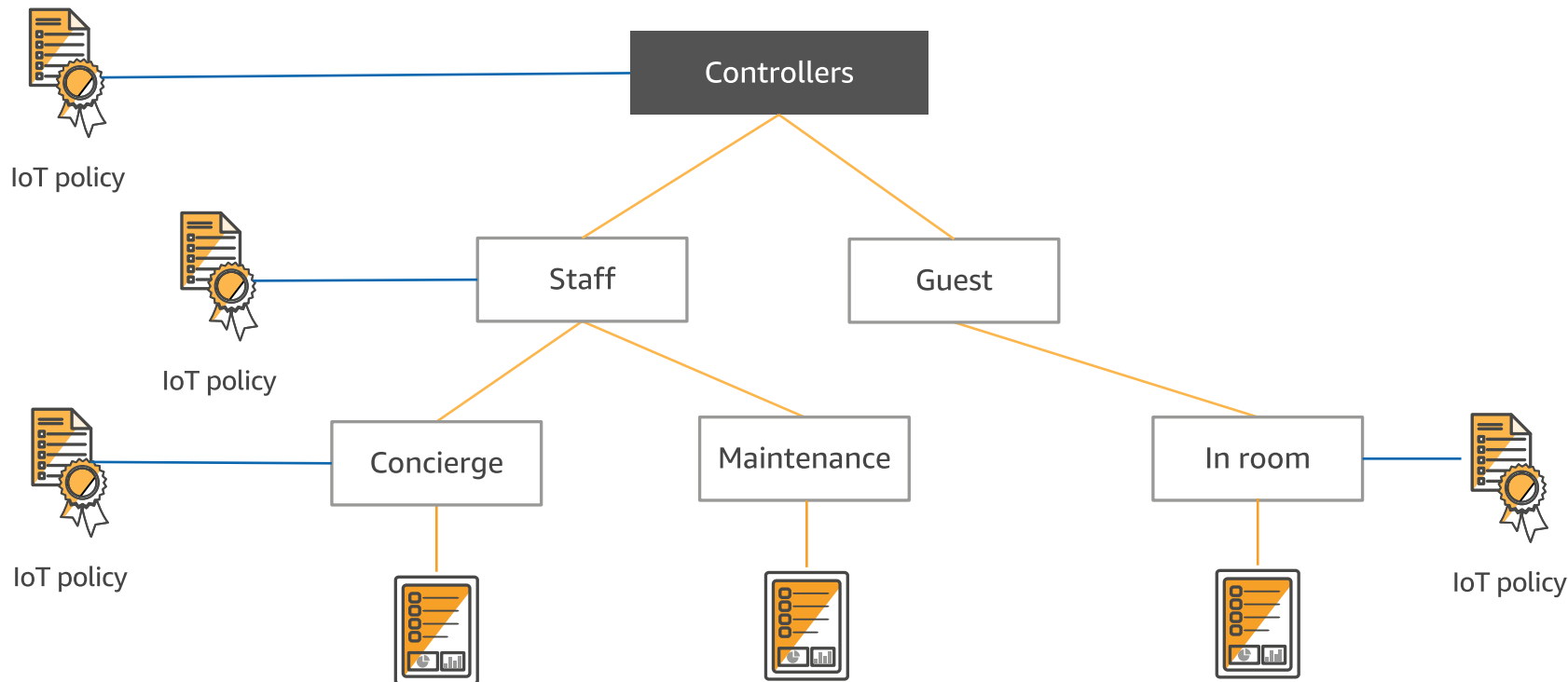


Thing Groups





Thing Group Policies





Thing Group Benefits

- Attaching Policies to groups facilitates device certificate rotation
- Assign jobs to groups instead to single devices
- Set logging level for a thing group. Useful for debugging because global log level remains unchanged



Device Registry/Shadow/Connectivity Indexing

Lucene-index queries

Get me all the widgets prior to version 6 that support more than 3 colors

```
"attributes.version < 6 AND attributes.colors > 3"
```

Find all rooms where the temperature is above 20°

```
"shadow.reported.temperature>20 AND attributes.room_number:"
```

Find all my lightbulbs that are currently on and no one present in the warehouse

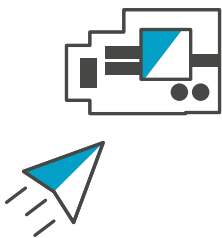
```
"shadow.reported.illumination_state = on AND shadow.reported.occupant present  
= no AND attributes.building_type = warehouse"
```

Find all connected devices

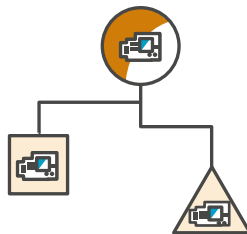
```
"connectivity.connected:true"
```



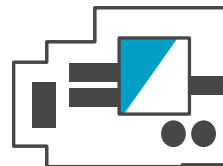
Device Notifications through Registry Events



Jobs



Groups



Devices



Registry Events

- AWS IoT publishes event messages when certain events occur
- Event messages are published over MQTT with a JSON payload
- Registry events for things, thing types, thing groups

Use-Cases

- Trigger Rules based on changes in the device registry
- Update own datastore when devices are CRUD
- Enrich data in the device registry

Access

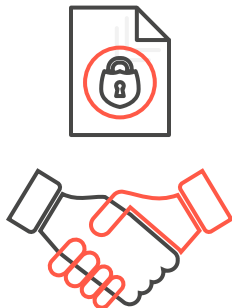


Secure Tunneling AWS IoT Device Management

Provides secure connectivity to individual devices in just a few clicks to diagnose issues and take action to solve them.



Gain remote access to devices on isolated networks or behind firewalls



Establish trusted connections that adhere to customers' corporate security policies



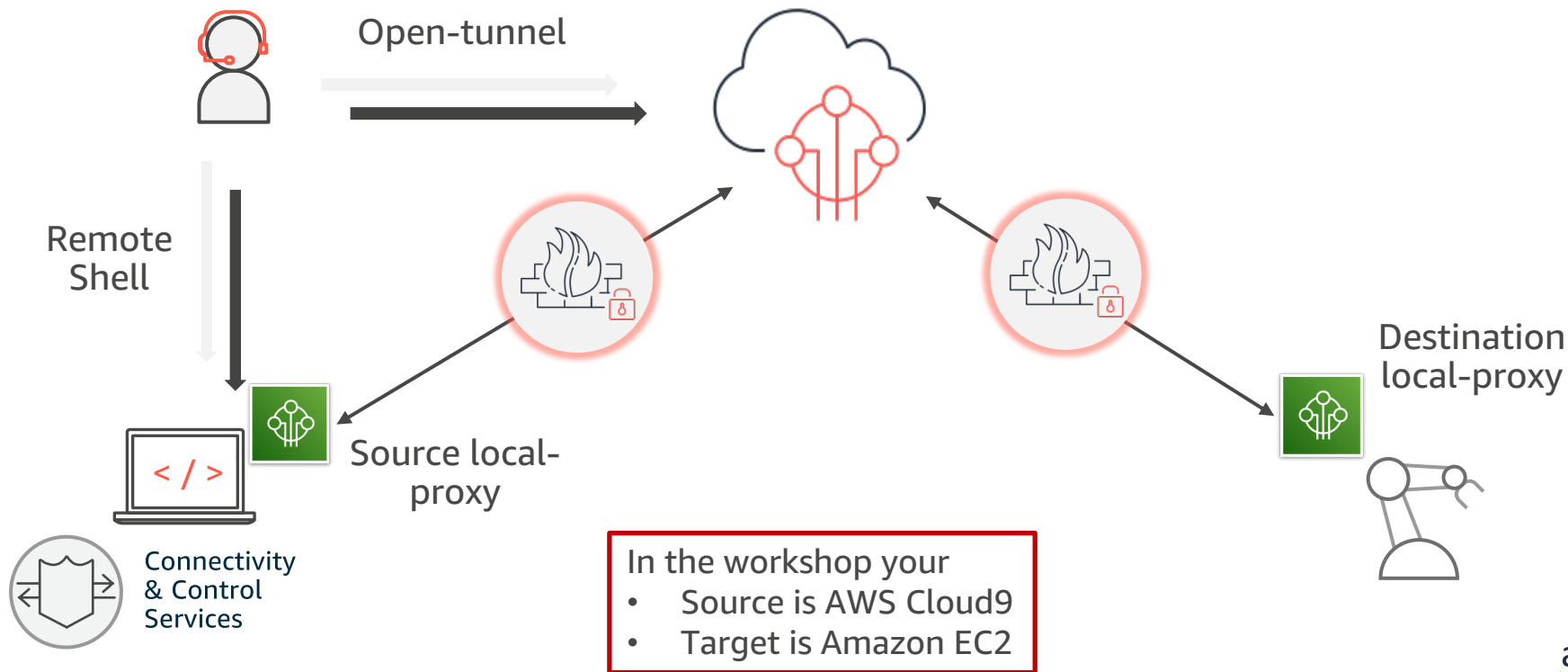
Troubleshoot and solve device issues more quickly and cost-effectively, with no disruption to end user experience



Connectivity
& Control
Services



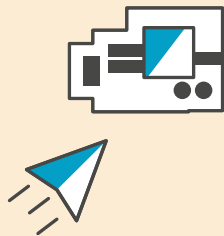
Secure Tunneling



Monitor

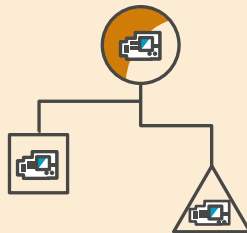


Monitoring Device Events



Jobs

Monitoring of
Device
Updates



Groups

Monitor Devices
Joining Groups



Policies

Monitor Device Security
Policies



Ressource-specific Logging

Groups /

logging-debug-group

Details

Resource-specific logging

Groups

You can choose to override the log level for this resource.

Things

Security

Local logging setting

Set to DEBUG

ENABLED

Jobs

Logs

Global logging setting

Set to WARN

ENABLED

```
{
  "timestamp": "2018-04-17 13:50:21.616",
  "logLevel": "INFO",
  "traceId": "6753a942-92c3-f979-587c-9c634874b672",
  "accountId": "123456789012",
  "status": "Success",
  "eventType": "Publish-In",
  "protocol": "MQTT",
  "topicName": "$aws/things/job-agent/jobs/get",
  "clientId": "job-agent",
  "principalId":
  "9187849467e75a1a92cbcf0f3a6a49b4f10d820b99dfa62657cf4b6e60c0dac4",
  "sourceIp": "35.178.51.181",
  "sourcePort": 46435
}
```

Update

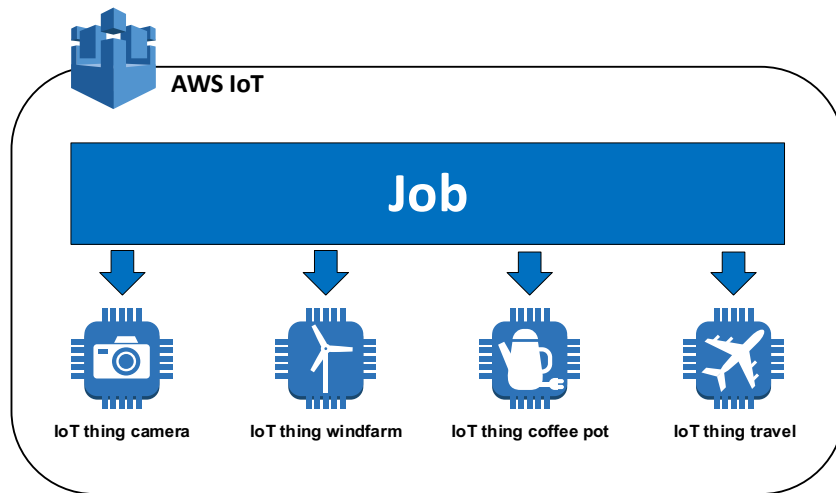


Define Local Actions Using Jobs

Jobs use JSON files called Job Documents to define actions that the device should take locally

Example use cases:

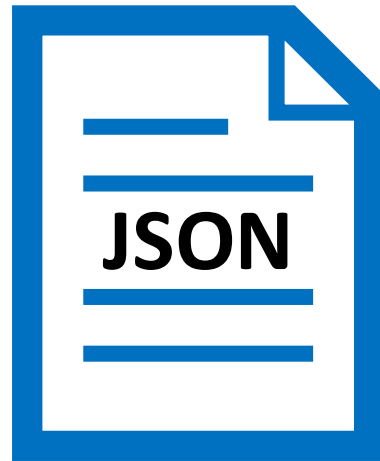
- Firmware updates
- Reboot a device
- Rotate certificates





Define Local Actions Using Jobs

- Include one or more locations of dependent data to download (i.e. S3 Objects)
- Use location links as placeholders for pre-signed URL at run-time
- JSON Encoded
- Create jobs using AWS Console, CLI, and SDK





Structure of Job Documents

```
{  
  "operation" : "reboot"  
}  
  
{  
  "operations" : {  
    "reboot" : "safe-mode",  
    "configurations" : {  
      "log" : "persist",  
      "download" : {  
        "target" : "${aws:iot:s3-presigned-  
url:https://s3.amazonaws.com/bucket/key}",  
        "patch" : "critical"  
      },  
      "restart" : "blemodule"  
    }  
  }  
}
```

Builder's Time

<http://bit.ly/aws-iot-device-management-workshop>

Any Questions?



Thank you!
Please complete the survey
(link at the website)

